

```
private void privateMethod()
{
    iamprivate = 2;
    System.out.println("" + iamprivate);
}
}
```

The Beta class, for example, cannot access the `iamprivate` variable or invoke `privateMethod` on an object of type Alpha because Beta is not of type Alpha.

```
class Beta
{
    public void accessMethod()
    {
        Alpha a = new Alpha();
        a.iamprivate = 10; // Invalid
        a.privateMethod(); // Invalid
    }
}
```

The compiler would complain if you tried this. So can one instance of an Alpha object access the private data variables of another instance of an Alpha object? Well yes objects of the same type have access to one another's private members.

5.2.2 Package

If you do not specify the access for either a method or an encapsulated data variable, then it is given the default access: package. This access allows other classes in the same package as your class to access its data variables as if they were their own. This level of access assumes that classes in the same package as your class are friends who won't harm your class' data.

Notice that no access modifier is declared. So, `iamprivate` and method `privateMethod` both default to package access. All classes declared in the package Greek, along with class Delta, have access to `iamprivate` and `privateMethod`.

```
package Greek;
class Delta
{
```